📝 DAY 2: STUDENT QUICK REFERENCE & OFFICIAL MCQ BANK

1-Page Cheat Sheet + 25 Evaluated Assessment Questions

Course Code: GLA-HPC-BCSG701 | **Track:** Intel & NEC Supercomputing Alliance
Primary Reference: `Intel & NEC - High Performance Computing.pdf`

📌 PART 1: 1-Page Student Command & Pragma Cheat Sheet

> **TIP**
>
> ### ⚡ 2-MINUTE INSTANT-RECALL MASTERY BOX (80-90% IMMEDIATE KNOWLEDGE)
>
> - **Amdahl's Speedup Ceiling:** $\text{Speedup} = \frac{1}{(1-P)+\frac{P}{N}}$. If 10% is serial ($1 - P = 0.10$), max possible speedup is $10\times$ even with infinite cores!
> - **OpenMP vs MPI Rule:** OpenMP runs inside 1 server (threads share RAM); MPI runs across separate servers (ranks pass network messages).
> - **False Sharing Elimination:** Never let adjacent threads write to adjacent array indices on the same 64-byte cache line; pad with `alignas(64)` for 4x-8x speedup.
> - **Top 5 Life-Saving Code Snippets:**
>   1. `#pragma omp parallel for reduction(+:total)` (Safely parallelize loop accumulation with zero lock overhead)
>   2. `export OMP_NUM_THREADS=8` (Control active OpenMP worker thread count from shell)
>   3. `mpicxx -O3 app.cpp -o app && mpirun -np 4 ./app` (Compile and launch 4 distributed MPI processes)
>   4. `MPI_Reduce(&local, &global, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD)` (Aggregate partial sums into Rank 0)
>   5. `from sklearnex import patch_sklearn; patch_sklearn()` (Accelerate Scikit-learn by 15x-35x using Intel oneDAL)

1. OpenMP Directives (Intra-Node Shared Memory)

Directive	What It Does	Example
`#pragma omp parallel`	Forks team of worker threads	`#pragma omp parallel { ... }`
`#pragma omp for`	Distributes loop iterations across threads	`#pragma omp parallel for`
`reduction(+:sum)`	Eliminates race condition on accumulators	`#pragma omp parallel for reduction(+:sum)`
`omp_get_thread_num()`	Returns thread ID (0 to N-1)	`int tid = omp_get_thread_num();`

Directive	What It Does	Example
`omp_get_max_threads()`	Returns total configured threads	`omp_get_max_threads()`

2. MPI Primitives (Inter-Node Distributed Memory)

Primitive	What It Does	Scope
`MPI_Init` / `MPI_Finalize`	Initializes / shuts down MPI execution	Process boundary
`MPI_Comm_rank`	Assigns unique ID (Rank) to each process	Per process
`MPI_Send` / `MPI_Recv`	Point-to-point data transmission	2 processes
`MPI_Bcast`	Broadcasts buffer from one rank to all ranks	Collective
`MPI_Reduce` / `MPI_Allreduce`	Combines arrays across ranks and computes sum/max	Collective

3. Intel Extension for Scikit-Learn (`sklearnex`)

```
# Enable hardware acceleration dynamically:
from sklearnex import patch_sklearn
patch_sklearn()

# All standard Scikit-Learn imports now run on Intel oneDAL:
from sklearn.cluster import KMeans
from sklearn.svm import SVC
from sklearn.decomposition import PCA
```

🎯 PART 2: Day 2 Official Syllabus Checklist

- ▢ 01. Parallel computing principles & Amdahl's Law
- ▢ 02. Task parallelism vs Data parallelism
- ▢ 03. Shared-memory architecture vs Distributed-memory architecture
- ▢ 04. OpenMP Fork-Join execution model
- ▢ 05. OpenMP compiler pragmas (`#pragma omp parallel for`)
- ▢ 06. Data sharing clauses: `shared` , `private` , `firstprivate`
- ▢ 07. Race conditions and reduction operations (`reduction(+:sum)`)
- ▢ 08. Thread scheduling: `static` vs `dynamic`
- ▢ 09. MPI communicator, rank, and size concepts
- ▢ 10. MPI point-to-point communication (`MPI_Send` , `MPI_Recv`)
- ▢ 11. MPI collective operations (`MPI_Bcast` , `MPI_Scatter` , `MPI_Reduce`)
- ▢ 12. Intel AI Analytics Toolkit components overview
- ▢ 13. Intel oneMKL integration with NumPy and Pandas
- ▢ 14. Intel Extension for Scikit-learn (`sklearnex`) dynamic patching
- ▢ 15. Accelerated machine learning algorithms: k-Means, SVM, PCA, Random Forest

📝 PART 3: 25 Official Multiple-Choice Questions (With Answers & Explanations)

Q1. According to Amdahl's Law, if 20% of an application cannot be parallelized, what is the maximum theoretical speedup possible with an infinite number of processors?

- A) 2x
- B) 5x
- C) 20x
- D) Infinite
  Answer: B | *Explanation:* $\text{Speedup} = 1/(1-P) = 1/0.20 = 5x$.

Q2. Which execution model describes OpenMP multithreading?

- A) Master-Worker Queue only
- B) Fork-Join Model
- C) Peer-to-Peer Actor Model
- D) Client-Server Architecture
  Answer: B | *Explanation: OpenMP master thread forks worker threads upon encountering a parallel region and joins at the barrier.*

Q3. What is a "Data Race" in shared-memory parallel programming?

- A) A competition to download datasets faster
- B) When two threads access the same memory location concurrently, and at least one is a write, without synchronization
- C) When a network cable is disconnected
- D) When memory runs out on the disk
  Answer: B | *Explanation: Unsynchronized concurrent memory accesses where at least one is a mutation produce non-deterministic data races.*

Q4. Which OpenMP clause is used to safely sum numbers across threads without race conditions?

- A) `private(sum)`
- B) `reduction(+:sum)`
- C) `shared(sum)`
- D) `flush(sum)`
  Answer: B | *Explanation: The reduction clause creates thread-private accumulators and combines them atomically at the barrier.*

Q5. What is the fundamental difference between OpenMP and MPI?

- A) OpenMP is for Python; MPI is for Java
- B) OpenMP targets shared-memory intra-node cores; MPI targets distributed-memory inter-node systems over networks
- C) OpenMP is obsolete; MPI is modern
- D) There is no difference
  Answer: B | *Explanation: OpenMP operates over shared system RAM; MPI communicates across independent nodes over network fabrics.*

Q6. What does `MPI_Comm_rank` return?

- A) The total number of nodes in the cluster
- B) The unique integer ID assigned to the calling process
- C) The CPU clock speed
- D) The network IP address
  Answer: B | *Explanation: Rank uniquely identifies each process within an MPI communicator (from 0 to size-1).*

Q7. Which MPI collective routine sends data from one root rank to all other ranks?

- A) `MPI_Send`
- B) `MPI_Bcast`
- C) `MPI_Reduce`
- D) `MPI_Barrier`
  Answer: B | *Explanation: `MPI_Bcast` broadcasts a buffer from the designated root process to all processes in the communicator.*

Q8. What two lines of Python code activate Intel oneDAL acceleration for Scikit-Learn?

- A) `import onedal; onedal.speedup()`
- B) `from sklearnex import patch_sklearn; patch_sklearn()`
- C) `import intel_ai; intel_ai.boost()`
- D) `import sklearn; sklearn.accelerate()`
  Answer: B | *Explanation: `patch_sklearn()` replaces standard Scikit-Learn classes with optimized oneDAL counterparts.*

Q9. Which algorithm achieves massive performance gains when accelerated by `sklearnex`?

- A) Support Vector Machines (SVM) & k-Means
- B) Linear search loops
- C) JSON file parsing
- D) Regular expression parsing
  Answer: A | *Explanation: Mathematical algorithms like k-Means, SVM, and PCA map directly onto Intel oneDAL vectorized SIMD routines.*

Q10. Which compiler flag enables OpenMP support in the Intel oneAPI C++ Compiler (`icpx`)?

- A) `-fopen`
- B) `-qopenmp`
- C) `-parallel`
- D) `-threads`
  Answer: B | *Explanation: Intel compilers use `-qopenmp` (or `-fopenmp` in modern LLVM drivers) to activate OpenMP directives.*

Q11. What is "False Sharing" in multi-threaded CPU architectures?

- A) Sharing incorrect passwords
- B) When two CPU cores concurrently modify independent variables located on the SAME 64-byte cache line, causing continuous cache invalidations
- C) Copying files over slow Wi-Fi
- D) Running two Python scripts at once
  Answer: B | *Explanation: False sharing triggers the hardware MESI cache coherence protocol, bouncing cache lines*

between cores and degrading throughput.

Q12. In OpenMP, what does `schedule(static, chunk_size)` do?

- A) Spawns new threads dynamically during runtime
- B) Deterministically assigns fixed contiguous blocks of loop iterations to threads before execution
- C) Prevents thread execution
- D) Randomizes loop iterations
  Answer: B | *Explanation: Static scheduling assigns loop iterations in predetermined chunks, optimal for balanced workloads.*

Q13. Which library powers the mathematical acceleration in Intel AI Analytics Toolkit for NumPy operations?

- A) Pygame
- B) Intel oneMKL (Math Kernel Library)
- C) TensorFlow
- D) OpenCV
  Answer: B | *Explanation: oneMKL accelerates matrix multiplication (GEMM) and linear algebra across AVX-512 registers.*

Q14. What is the scope of an MPI communicator such as `MPI_COMM_WORLD`?

- A) It encompasses all processes spawned during `mpirun`
- B) It connects to the global public internet
- C) It only includes the master thread
- D) It refers to the hard disk partition
  Answer: A | *Explanation: `MPI_COMM_WORLD` is the default communicator containing all processes launched in the job.*

Q15. In an OpenMP parallel region, what is the default sharing attribute of variables declared OUTSIDE the parallel block?

- A) `private`
- B) `shared`
- C) `threadprivate`
- D) `deleted`
  Answer: B | *Explanation: Unless specified otherwise with `default(none)`, pre-existing variables are shared by default in C/C++.*

Q16. Which MPI collective gathers data from all ranks and applies an associative reduction operation (like summation or maximum)?

- A) `MPI_Gather`
- B) `MPI_Reduce`
- C) `MPI_Scatter`
- D) `MPI_Sendrecv`
  Answer: B | *Explanation: `MPI_Reduce` applies a mathematical reduction across elements from all ranks into a single root rank.*

Q17. How does Intel oneDAL accelerate classical machine learning algorithms?

- A) By rewriting Python code in JavaScript
- B) By compiling algorithms into native C++ kernels tuned for AVX-512 vector units and multi-core CPU cache topologies

- C) By sending data to a paid cloud API
- D) By downsampling the dataset
  Answer: B | *Explanation: oneDAL executes hand-tuned assembly and vector instructions matching the host CPU's exact microarchitecture.*

Q18. What command launches an MPI application across 8 parallel processes?

- A) `run -n 8 ./program`
- B) `mpirun -np 8 ./program`
- C) `exec 8 ./program`
- D) `start -np 8 ./program`
  Answer: B | *Explanation: `mpirun -np 8` (or `mpiexec -n 8`) launches 8 parallel MPI ranks.*

Q19. In OpenMP, what directive ensures that a block of code is executed by ONLY ONE thread at a time?

- A) `#pragma omp barrier`
- B) `#pragma omp critical`
- C) `#pragma omp single`
- D) `#pragma omp master`
  Answer: B | *Explanation: `#pragma omp critical` provides mutually exclusive serialization for sensitive code blocks.*

Q20. What is the typical speedup range achieved by `patch_sklearn()` on CPU-bound algorithms like k-Means or SVM?

- A) 1.05x
- B) 10x to 30x+
- C) No speedup
- D) 1000x
  Answer: B | *Explanation: Moving from single-threaded Python bytecode to native multi-core AVX-512 oneDAL delivers 10x to 30x+ speedups.*

Q21. Which function returns the execution time in seconds with high resolution in OpenMP?

- A) `time()`
- B) `omp_get_wtime()`
- C) `clock()`
- D) `gettimeofday()`
  Answer: B | *Explanation: `omp_get_wtime()` returns wall-clock time in seconds as a high-resolution double.*

Q22. What is "Task Parallelism" as contrasted with "Data Parallelism"?

- A) Processing numbers vs processing text
- B) Distributing different concurrent functions/tasks across cores vs distributing chunks of the same dataset across cores
- C) Parallelism on GPUs vs CPUs
- D) Running jobs on Windows vs Linux
  Answer: B | *Explanation: Task parallelism executes distinct threads of execution; data parallelism executes the same instruction over partitioned data.*

Q23. In an MPI point-to-point communication, what must match between `MPI_Send` and `MPI_Recv`?

- A) The message tag, communicator, and datatype
- B) The process rank must be identical

- C) The file permissions
- D) The CPU clock speed
  Answer: A | *Explanation: Messages are matched by communicator, destination/source rank, and integer message tag.*

Q24. Does `patch_sklearn()` require modifying existing machine learning model hyperparameters?

- A) Yes, all parameters must be retrained
- B) No, it maintains 100% Scikit-Learn API parity and mathematical equivalence
- C) Yes, it only works with 3 clusters
- D) Yes, it requires converting data to images
  Answer: B | *Explanation: `sklearnex` conforms strictly to standard Scikit-Learn APIs while accelerating execution under the hood.*

Q25. What supercomputing topic builds directly upon Day 2 parallel architectures on Day 3?

- A) Web development with React
- B) Deep Learning with TensorFlow-MKL, oneDNN, and Slurm cluster batch scheduling
- C) Database administration with SQL
- D) Excel formulas
  Answer: B | *Explanation: Day 3 scales the parallel execution principles of Day 2 into Deep Learning frameworks and cluster batch managers.*